

Estação 1

De caracteres a lexemas e tokens

JMain.java × +

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 public class Main {
5     static final String ALUNO = "GUSTAVO CARDOSO" ;
6     static final String RA = "240111752";
7
8     //static final String ENTRADA = "resultado = somaAnterior - valor / 100;";
9     static final String ENTRADA = "total_gu = valor + 52;";
10    // Troque mg por suas iniciais e 25 pelos dois últimos dígitos do RA.
11
12    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
13
14    static {
15        SIMBOLOS.put('=', "ATRIBUICAO");
16        SIMBOLOS.put('+', "SOMA");
17        SIMBOLOS.put('-', "SUBTRACAO");
18        SIMBOLOS.put('*', "MULTIPLICACAO");
19        SIMBOLOS.put('/', "DIVISAO");
20        SIMBOLOS.put(';', "PONTO_E_VIRGULA");
21        SIMBOLOS.put('(', "ABRE_PARENTESES");
22        SIMBOLOS.put(')', "FECHA_PARENTESES");
23    }
24
25    public static void main(String[] args) {
26        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
27        System.out.println("ENTRADA: " + ENTRADA);
28        System.out.println();
29    }
30 }
```

Console I/O AI Agent

ALUNO: GUSTAVO CARDOSO | RA: 240111752
ENTRADA: total_gu = valor + 52;

LEXEMA	TOKEN
total_gu	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
52	INTEIRO
;	PONTO_E_VIRGULA

Estação 2

O que o analisador léxico reconhece

=== TESTE C2 - identificador personalizado ===

ENTRADA: int nota_GU = 10;

LEXEMA	CLASSE	TOKEN
int	LETRA	PALAVRA_RESERVADA
nota_GU	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===

ENTRADA: int valor# = 1;

LEXEMA	CLASSE	TOKEN
int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

Estação 3

Quando o compilador encontra erros

JMain.java x+

ConsoleI/OAI AgentExit 1

1public class Main {
2static final String ALUNO = "GUSTAVO CARDOSO";
3static final String RA = "240111752";
4
5public static void main(String[] args) {
6System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e vírgula
9
10if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto ent
12System.out.println("Resultado: " + total);
13}
14}
15}
16}

Main.java:8: error: ';' expected
int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e vírgula
^
Main.java:10: error: ')' expected
if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
^
Main.java:11: error: unclosed string literal
System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o
texto entre aspas
^
3 errors

1public class Main {
2static final String ALUNO = "GUSTAVO CARDOSO";
3static final String RA = "240111752";
4
5public static void main(String[] args) {
6System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e vírgula
9
10if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto en
12System.out.println("Resultado: " + total);
13}
14}
15}
16}

ALUNO: GUSTAVO CARDOSO | RA: 240111752
Programa corrigido!
Resultado: 30

Parser descendente recursivo

ALUNO: GUSTAVO CARDOSO | RA: 240111752

ENTRADA: $(2 + 3) * 4$

```
ENTRANDO em <expr> | lookahead = (  
  ENTRANDO em <term> | lookahead = (  
    ENTRANDO em <factor> | lookahead = (  
      Abrindo parenteses  
      Novo lookahead: 2  
      ENTRANDO em <expr> | lookahead = 2  
        ENTRANDO em <term> | lookahead = 2  
          ENTRANDO em <factor> | lookahead = 2  
            Numero reconhecido: 2  
            Novo lookahead: +  
            SAINDO de <factor> | lookahead = +  
          SAINDO de <term> | lookahead = +  
        Encontrado operador +  
        Novo lookahead: 3  
      ENTRANDO em <term> | lookahead = 3  
        ENTRANDO em <factor> | lookahead = 3  
          Numero reconhecido: 3  
          Novo lookahead: )  
        SAINDO de <factor> | lookahead = )  
      SAINDO de <term> | lookahead = )  
    SAINDO de <expr> | lookahead = )  
  Novo lookahead: *  
)
```

Parser ascendente: deslocar e reduzir

ALUNO: GUSTAVO CARDOSO | RA: 240111752
ENTRADA: id+id*id

PILHA	ENTRADA	ACAO

\$ 0	id+id*id\$	SHIFT id -> estado 5
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + -> estado 6
\$ 0 E 1 + 6	id*id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T * F
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E + T
\$ 0 E 1	\$	ACCEPT